



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA
INGENIERÍA EN SISTEMAS DE INFORMACIÓN

ANÁLISIS DE DATOS EMPRESARIALES

Trabajo Práctico Final

Grupo 2 - Escenario Discográfica

Profesor:

- Ing. Andrés Pablo **Fantin**.

Alumnos:

- Aldo Omar **Andres**.
- Sixto Feliciano **Arrejin**.
- Agustín Nicolás **Bravo Pérez**.
- Nilson **Orrego**.
- André Leandro **San Lorenzo**.

30 de Abril de 2025

Índice

Consigna.....	1
Etapa 1: Diseño Conceptual.....	1
Descripción del problema y contexto.....	1
Objetivos y Consultas.....	3
Modelo Conceptual Multidimensional.....	3
Granularidad de la Información.....	6
Etapa 2: Diseño Lógico.....	7
Diagrama Conceptual.....	7
Restricciones No Modeladas.....	8
Slow Changing Dimensions (SCD).....	8
Etapa 3: Diseño Físico.....	9
Script SQL de Creación de Tablas.....	10

Consigna

El proyecto del curso debe ser entregado en un solo archivo pdf por este medio. INCLUYA EL ESCENARIO DADO y los considerando para mayor practicidad a la hora de corregir.

Los entregables del proyecto son:

Etapa 1: Diseño Conceptual

1. Descripción del problema, contexto (entrega la cátedra).
2. Objetivos, subobjetivos, y consultas, a partir de las cuales se definen las dimensiones, facts, etc.
3. Diseño conceptual usando la metodología "multidim" vista en teoría y además:
 - a. Explique el proceso que realizó para analizar este escenario y cómo llegó el grupo al modelo multidimensional conceptual donde almacenará esta información detallando medidas de la tabla de hechos que recolectarán y atributos complementarios.
 - b. Determinar y fundamentar la granularidad de la información a almacenar.

Etapa 2: Diseño Lógico

4. Especifique en el diagrama los atributos de las tablas de hechos y de dimensión.
5. Detalle todas las restricciones del diseño conceptual que no pudo reflejar en el modelo lógico y cómo las resolverá más adelante.
6. Tenga en cuenta el problema de "slow changing dimension" en el diseño de las dimensiones, preparando el mismo para que, ante algún cambio en los atributos de estas, pueda mantener la información histórica.

Etapa 3: Diseño Físico

7. Creación del datawarehouse en el motor de base de datos relacional elegido por el grupo - Se deberá entregar el script SQL de creación como parte del informe. NO IMPORTE NINGÚN DATO AUN. SOLO ESQUEMA DE ENTIDADES VACÍO.

Etapa 1: Diseño Conceptual

Descripción del problema y contexto

Un sello discográfico quiere hacer un seguimiento de todos los **contratos** que tiene con bandas, los **discos** que está produciendo y las **ventas** de dichos discos. Actualmente, solo mantienen datos de los contratos vigentes; no se conserva información histórica.

Por ello, se decide construir un **data warehouse** para recolectar y almacenar esa información histórica.

Con el **data warehouse**, la empresa desea analizar sus ventas.

Basado en conversaciones con los gerentes de la empresa, se pudo recopilar la siguiente descripción de los datos disponibles:

- El sello discográfico tiene varias bandas bajo contrato.
- Cada banda tiene un nombre, una **categoría principal** de música y uno o más artistas.
- Una banda **puede tener un manager**, pero no es obligatorio. Las bandas sin manager deben tener a uno de sus artistas como **representante principal**.
- Un artista tiene un **instrumento principal**, que también puede ser su voz.
- Con el tiempo, las bandas pueden separarse o cambiar de miembros.
- Las bandas graban discos, que son producidos y distribuidos por el sello discográfico.
- Los discos, incluso los que aún no están terminados, tienen un título y están identificados por un **código internacional** especial.
- Para cada disco, se almacena el **costo de producción** y la **fecha de lanzamiento**.

Además:

- El sello tiene **contratos** con las bandas. Para cada contrato se registra:
 - Fecha de inicio y fin.
 - Discos que abarca.
 - Porcentaje de ventas que va al manager (si lo hay), a la banda y al sello.
 - Un contrato puede aplicarse a varios discos, pero **cada disco pertenece a un solo contrato**.
- Las **tiendas** pueden pedir discos al sello. Se registra:
 - Fecha del pedido, cantidad, precio acordado.
 - Cantidad finalmente enviada (la demanda puede superar la oferta).
 - Fecha en que se cumplió el pedido.
 - Información de la tienda: ciudad, país y región.

Ejemplos de análisis que el sello desea hacer:

- ¿Qué disco / grupo / artista ha generado la mayor o menor ganancia o retorno de inversión, por semana / mes / año?
- ¿Qué vendedor / ciudad / país tiene la mayor cantidad de demandas no satisfechas?
- ¿Qué disco / grupo / artista tiene la mayor cantidad de pedidos?

Consideraciones:

- No ubicar atributos dimensionales incorrectamente en jerarquías (por ejemplo: hacer roll-up de banda a disco).
- No modelar relaciones muchos-a-muchos como si fueran uno-a-muchos.
- No usar solo una sola tabla de hechos cuando en realidad hay más de un tema de análisis (por ejemplo: los contratos no son una propiedad de las ventas, sino dos hechos separados).
- Se podría considerar un tercer hecho para representar la producción del disco.

Objetivos y Consultas

En base al escenario, proponemos el siguiente objetivo general:

Proveer un data warehouse con datos históricos que permita al sello discográfico comprender, medir y optimizar sus actividades de **producción y ventas de discos**, facilitando la toma de decisiones estratégicas y operativas.

Objetivos específicos:

1. Analizar el desempeño comercial.
2. Evaluar la rentabilidad del negocio.
3. Evaluar la efectividad de los contratos.
4. Gestionar la demanda y optimizar la distribución de discos a tiendas.

Listado de consultas que el data warehouse deberá poder responder:

1. ¿Cuál disco / contrato / banda / categoría ha generado la mayor cantidad de ingresos por día / mes / año?
2. ¿Cuál tienda / ciudad / país realiza la mayor cantidad de ventas?
3. ¿Cuál tienda / ciudad / país tiene la mayor demanda insatisfecha?
4. ¿Qué disco / banda / categoría tiene la mayor cantidad de pedidos?
5. ¿Cuál disco fue el más costoso de producir en cierto mes / año?
6. ¿Qué artista está asociado a los discos más vendidos?
7. ¿Qué banda generó más ingresos este trimestre / año?
8. ¿Qué porcentaje de contratos tienen un manager de la banda asociado?
9. ¿Cuál fue el costo de producción por disco?
10. ¿Cuál fue la duración promedio entre el inicio de la producción y el lanzamiento del disco?

Modelo Conceptual Multidimensional

A continuación explicamos el análisis del escenario que realizamos para llegar al modelo multidimensional conceptual.

Primero identificamos que la necesidad más importante era analizar los aspectos financieros del escenario. Esto significa calcular los ingresos por las **ventas** y los

egresos por los **contratos** y las **producciones** de discos. Estos son los tres hechos del data warehouse.

Las siguientes dimensiones dan contexto a los hechos:

- **Tiempo:** para analizar los hechos en el tiempo (fecha de pedido y entrega de una venta, fecha de inicio y fin de una producción, fecha de inicio y finalización de los contratos, fecha de lanzamiento del disco).
- **Tienda:** para agrupar las ventas.
- **Ubicación:** para ubicar las tiendas, y agrupar ventas por ubicación geográfica. Según el escenario, nos interesa la ciudad, región y país.
- **Disco:** para analizar las ventas por cada disco, para por ejemplo conocer los discos más vendidos.
- **Categoría:** para agrupar las ventas por género musical.
- **Banda:** para agrupar ventas (ingresos) y producciones (costos) por banda.
- **Artista:** una banda está compuesta por varios artistas.
- **Instrumento:** para categorizar a los artistas según el instrumento que tocan.
- **Manager:** el manager de una banda se lleva un porcentaje de las ventas del disco, lo cual está definido en el contrato.

Por ende, los hechos y sus medidas son:

- Ventas:
 - Precio unitario.
 - Precio total.
 - Cantidad pedida: número de discos solicitados en un pedido.
 - Cantidad vendida: número de discos finalmente entregados.
 - Hay demanda insatisfecha: indica si la cantidad pedida es mayor a la cantidad vendida.
- Contratos:
 - Porcentaje Banda: porcentaje de las ventas que recibe la banda.
 - Porcentaje Manager: porcentaje de las ventas que recibe el manager.
 - Porcentaje Sello: porcentaje de las ventas que retiene el sello.
 - Costo Contrato: lo que pagó la discográfica para realizar el contrato.
- Producciones:
 - Costo de producción: el gasto incurrido en la creación del disco.
 - Duración: diferencia entre la fecha de inicio y la fecha de fin.

Dimensiones y sus atributos:

- Tiempo:
 - Fecha completa.
 - Nombre del día.
 - Día del mes.
 - Día de la semana.
 - Semana.
 - Mes.

- Trimestre.
 - Año.
 - Es fin de semana.
 - Es día laboral.
- Tienda:
 - ID tienda.
 - Nombre tienda.
 - Dirección.
- Ubicación:
 - ID ubicación.
 - Ciudad.
 - País.
 - Región.
- Disco:
 - ID disco.
 - Código internacional.
 - Título.
- Categoría:
 - ID categoría.
 - Nombre categoría.
- Instrumento:
 - ID instrumento.
 - Nombre instrumento.
 - Familia.
 - Tipo instrumento.
- Artista:
 - ID artista.
 - Nombre artístico.
 - Nombre apellido.
 - Fecha nacimiento.
 - País origen.
 - Es activo.
 - Es solista.
- Banda:
 - ID banda.
 - Nombre banda.
 - Cantidad artistas.
 - País origen.
 - Es activa.
- Manager
 - ID manager.
 - Nombre completo.
 - Teléfono.
 - País origen.
 - Es activo.

- DiscoContrato: tabla puente para resolver la relación inconveniente en la cual un Contrato (hecho) tiene muchos Discos (dimensión). Esta tabla debe tener atributos para que tenga sentido, pero no logramos identificar atributos realmente adecuados.
 - Orden del disco: indica el orden de producción del disco en el contrato. Ejemplo: el segundo disco en un contrato de cuatro discos.

Granularidad de la Información

Granularidad de la información a almacenar en las tablas de hechos:

Ventas: granularidad por tienda, por ubicación (ciudad, región, país), por tiempo (día, semana, mes, trimestre, año), por disco, por banda, y por categoría. Esta granularidad posibilita responder preguntas como:

- ¿Qué tienda tiene la mayor demanda insatisfecha?
- ¿Cuál fue el precio promedio de venta de un disco en un pedido particular?
- ¿Cómo se compara la cantidad pedida con la cantidad finalmente enviada para un disco en una fecha específica?

Contratos: granularidad por tiempo y por banda. Dado que los contratos tienen costos asociados, agruparlos por tiempo nos permite conocer los costos de cierto período. Agruparlos por banda nos dice qué bandas son más caras de contratar.

Producciones: granularidad por tiempo y por banda. Cada disco tiene exactamente una producción asociada, y viceversa. Fundamentación similar a la de los contratos.

La granularidad por tiempo de estos tres hechos nos permite obtener las ganancias o pérdidas que tuvo la discográfica en cierto período.

Etapa 2: Diseño Lógico

Diagrama Conceptual

Diagrama conceptual: draw.io

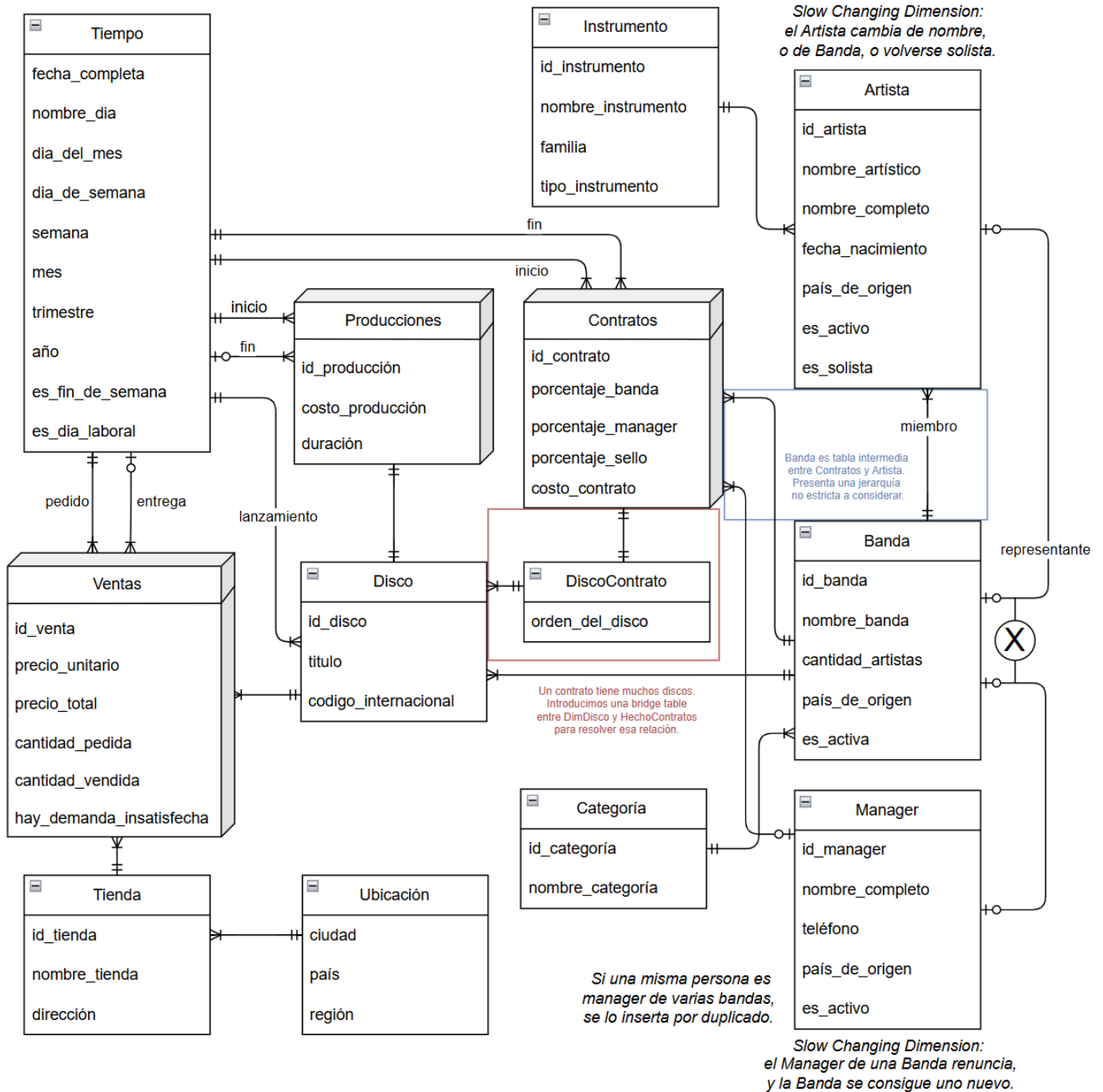


Figura 1. Diagrama relacional del Data Warehouse para la discográfica.

Restricciones No Modeladas

La siguiente restricción no se puede reflejar completamente en el modelo lógico: "Una banda puede tener un manager, pero no es obligatorio. Las bandas sin manager deben tener a uno de sus artistas como representante principal."

La exclusión mutua entre las dos claves foráneas opcionales (debe estar una o la otra, no ambas ni ninguna) se controlará con la siguiente restricción CHECK en la tabla DimBanda:

```
CONSTRAINT ManagerRepresentanteMutex CHECK ((dwid_manager IS NOT NULL AND dwid_representante IS NULL) OR (dwid_manager IS NULL AND dwid_representante IS NOT NULL))
```

Esta restricción CHECK se define más adelante, en el diagrama realizado en ER/Studio Data Architect, para ser considerada en el script SQL que esa herramienta genera.

Slow Changing Dimensions (SCD)

Identificamos dos dimensiones SCD:

- **Artista:** el artista puede cambiar de nombre, cambiar de Banda, o volverse solista. Necesitamos saber en qué período de tiempo el artista fue parte de la banda para saber a qué discos contribuyó.
- **Manager:** el manager de una Banda puede renunciar, pero puede ser relevante saber en qué período de tiempo fue parte de la banda. Similar si la banda consigue un nuevo manager.

La estrategia que adoptamos para mantener el historial de cambios es la SCD de **tipo 2**, que consiste en agregar una nueva fila a la dimensión cada vez que un atributo cambia. A las tablas se les agregan las siguientes columnas:

- **desde:** fecha de inicio de la validez de este registro.
- **hasta:** fecha de fin de la validez de este registro. Si es NULL, significa que es el registro actualmente válido. Si tiene un valor, significa que este registro es histórico.

Etapa 3: Diseño Físico

Utilizamos ER/Studio Data Architect como herramienta para crear el diseño lógico, especificando tipos de datos para cada columna de cada tabla.

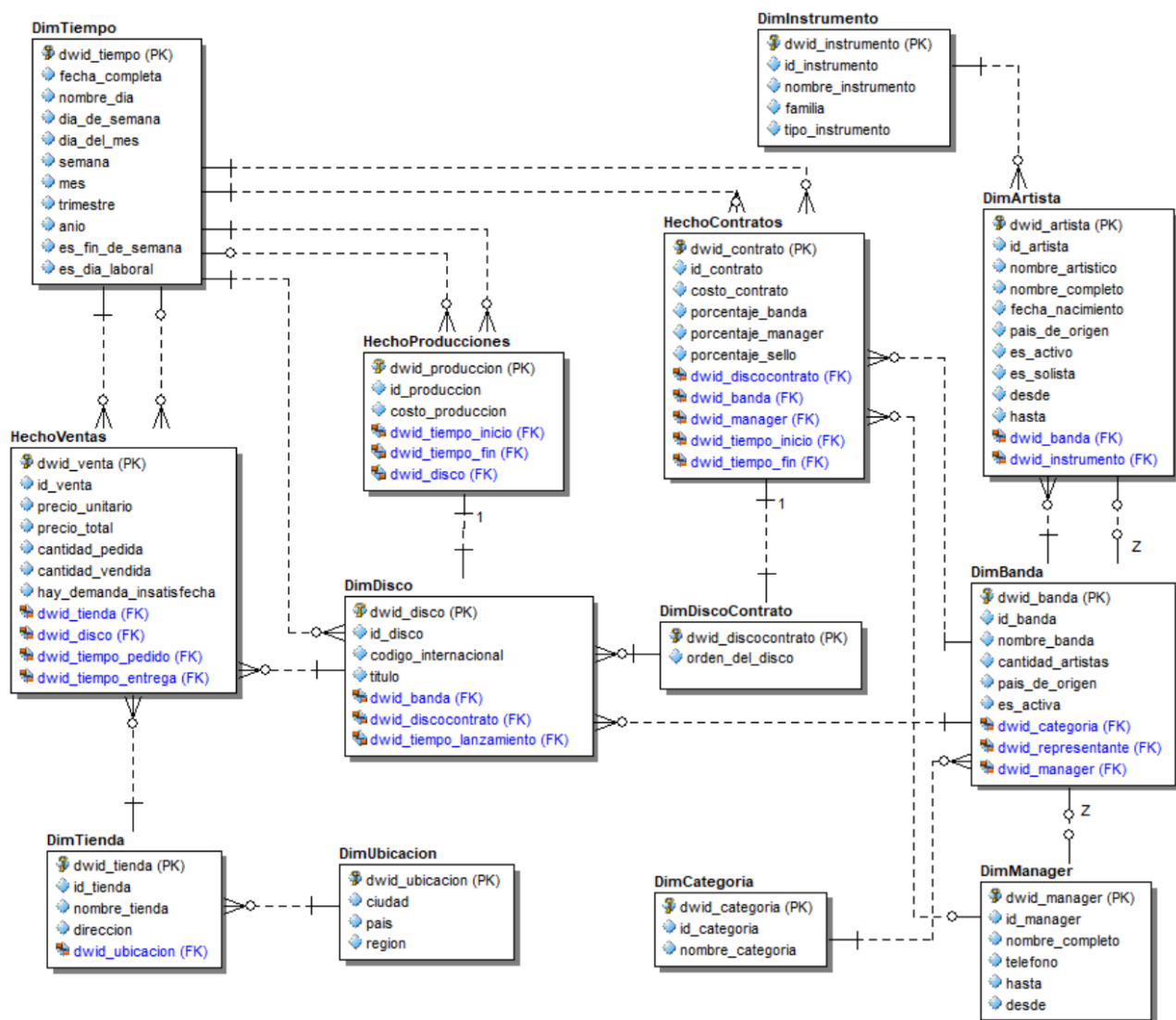


Figura 2. Diagrama del diseño lógico realizado en ER/Studio Data Architect.

Luego, en ER/Studio seleccionamos generar un archivo SQL con todas las sentencias de creación de tablas (DDL). Utilizaremos SQL Server 2022 como base de datos para implementar el Data Warehouse.

También utilizamos SQL Server Management Studio (SSMS) para crear una base de datos de nombre discografica. En SSMS podemos generar el siguiente diagrama de ingeniería inversa para validar las tablas:

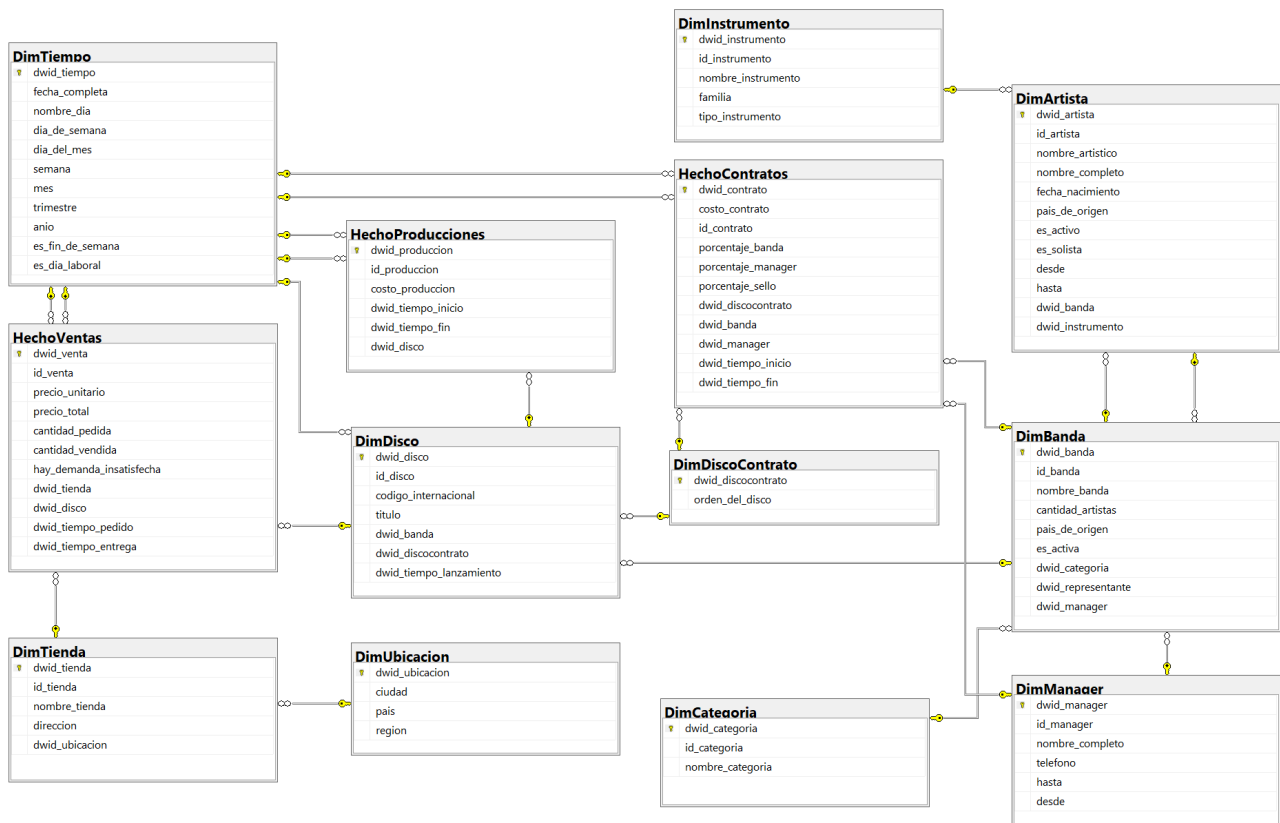


Figura 3. Diagrama generado por ingeniería inversa en SSMS, una vez ejecutado el script SQL.

Script SQL de Creación de Tablas

El siguiente script SQL fue generado mediante ER/Studio Data Architect. En el se pueden ver los tipos de dato que elegimos para cada columna.

```
/*
 * ER/Studio Data Architect SQL Code Generation
 * Project : Discografica.DM1
 *
 * Date Created : Tuesday, April 29, 2025 00:41:33
 * Target DBMS : Microsoft SQL Server 2022
 */
```

```
/*
 * TABLE: DimArtista
 */
```

```
CREATE TABLE DimArtista(
    dwid_artista    bigint    NOT NULL,
    id_artista      bigint    NOT NULL,
    nombre_artistico varchar(50) NOT NULL,
    nombre_completo varchar(50) NOT NULL,
    fecha_nacimiento date      NOT NULL,
    pais_de_origen  varchar(20) NOT NULL,
    es_activo       bit        NOT NULL,
    es_solista      bit        NOT NULL,
    desde           date       NOT NULL,
    hasta           date       NULL,
    dwid_banda      bigint    NOT NULL,
```

```

    dwid_instrumento    bigint    NOT NULL,
    CONSTRAINT PK18 PRIMARY KEY NONCLUSTERED (dwid_artista)
)

```

go

```

IF OBJECT_ID('DimArtista') IS NOT NULL
    PRINT '<<< CREATED TABLE DimArtista >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimArtista >>>'
go

```

```

/*
* TABLE: DimBanda
*/

```

```

CREATE TABLE DimBanda(
    dwid_banda          bigint    NOT NULL,
    id_banda            bigint    NOT NULL,
    nombre_banda        varchar(30) NOT NULL,
    cantidad_artistas   tinyint   NOT NULL,
    pais_de_origen      varchar(20) NOT NULL,
    es_activa           bit       NOT NULL,
    dwid_categoria      bigint    NOT NULL,
    dwid_representante  bigint    NULL,
    dwid_manager         bigint    NULL,
    CONSTRAINT ManagerRepresentanteMutex CHECK ((dwid_manager IS NOT NULL AND
dwid_representante IS NULL) OR (dwid_manager IS NULL AND dwid_representante IS NOT
NULL)),
    CONSTRAINT PK11 PRIMARY KEY NONCLUSTERED (dwid_banda)
)

```

go

```

IF OBJECT_ID('DimBanda') IS NOT NULL
    PRINT '<<< CREATED TABLE DimBanda >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimBanda >>>'
go

```

```

/*
* TABLE: DimCategoria
*/

```

```

CREATE TABLE DimCategoria(
    dwid_categoria      bigint    NOT NULL,
    id_categoria         bigint    NOT NULL,
    nombre_categoria    varchar(20) NOT NULL,
    CONSTRAINT PK20 PRIMARY KEY NONCLUSTERED (dwid_categoria)
)

```

go

```

IF OBJECT_ID('DimCategoria') IS NOT NULL
    PRINT '<<< CREATED TABLE DimCategoria >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimCategoria >>>'

```

go

```
/*  
 * TABLE: DimDisco  
 */
```

```
CREATE TABLE DimDisco(  
    dwid_disco          bigint      NOT NULL,  
    id_disco            bigint      NOT NULL,  
    codigo_internacional varchar(20) NOT NULL,  
    titulo              varchar(100) NOT NULL,  
    dwid_banda          bigint      NOT NULL,  
    dwid_discocontrato  bigint      NOT NULL,  
    dwid_tiempo_lanzamiento bigint  NOT NULL,  
    CONSTRAINT PK1 PRIMARY KEY NONCLUSTERED (dwid_disco)  
)
```

go

```
IF OBJECT_ID('DimDisco') IS NOT NULL  
    PRINT '<<< CREATED TABLE DimDisco >>>'  
ELSE  
    PRINT '<<< FAILED CREATING TABLE DimDisco >>>'  
go
```

```
/*  
 * TABLE: DimDiscoContrato  
 */
```

```
CREATE TABLE DimDiscoContrato(  
    dwid_discocontrato  bigint  NOT NULL,  
    orden_del_disco     tinyint NOT NULL,  
    CONSTRAINT PK9 PRIMARY KEY NONCLUSTERED (dwid_discocontrato)  
)
```

go

```
IF OBJECT_ID('DimDiscoContrato') IS NOT NULL  
    PRINT '<<< CREATED TABLE DimDiscoContrato >>>'  
ELSE  
    PRINT '<<< FAILED CREATING TABLE DimDiscoContrato >>>'  
go
```

```
/*  
 * TABLE: DimInstrumento  
 */
```

```
CREATE TABLE DimInstrumento(  
    dwid_instrumento  bigint  NOT NULL,  
    id_instrumento    bigint  NOT NULL,  
    nombre_instrumento varchar(20) NOT NULL,  
    familia           varchar(20) NOT NULL,  
    tipo_instrumento  varchar(20) NOT NULL,  
    CONSTRAINT PK19 PRIMARY KEY NONCLUSTERED (dwid_instrumento)  
)
```

go

```

IF OBJECT_ID('DimInstrumento') IS NOT NULL
    PRINT '<<< CREATED TABLE DimInstrumento >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimInstrumento >>>'
go

/*
* TABLE: DimManager
*/

CREATE TABLE DimManager(
    dwid_manager    bigint    NOT NULL,
    id_manager      bigint    NOT NULL,
    nombre_completo  varchar(50) NOT NULL,
    telefono         varchar(20) NOT NULL,
    hasta           date      NULL,
    desde           date      NOT NULL,
    CONSTRAINT PK12 PRIMARY KEY NONCLUSTERED (dwid_manager)
)

go

IF OBJECT_ID('DimManager') IS NOT NULL
    PRINT '<<< CREATED TABLE DimManager >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimManager >>>'
go

/*
* TABLE: DimTiempo
*/

CREATE TABLE DimTiempo(
    dwid_tiempo     bigint    NOT NULL,
    fecha_completa   date      NOT NULL,
    nombre_dia       varchar(10) NOT NULL,
    dia_de_semana    tinyint   NOT NULL,
    dia_del_mes      tinyint   NOT NULL,
    semana          tinyint   NOT NULL,
    mes             tinyint   NOT NULL,
    trimestre       tinyint   NOT NULL,
    anio            tinyint   NOT NULL,
    es_fin_de_semana bit       NOT NULL,
    es_dia_laboral   bit       NOT NULL,
    CONSTRAINT PK7 PRIMARY KEY NONCLUSTERED (dwid_tiempo)
)

go

IF OBJECT_ID('DimTiempo') IS NOT NULL
    PRINT '<<< CREATED TABLE DimTiempo >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimTiempo >>>'
go

```

```

/*
 * TABLE: DimTienda
 */

CREATE TABLE DimTienda(
    dwid_tienda    bigint    NOT NULL,
    id_tienda      bigint    NOT NULL,
    nombre_tienda  varchar(40) NOT NULL,
    direccion      varchar(40) NOT NULL,
    dwid_ubicacion bigint    NOT NULL,
    CONSTRAINT PK5 PRIMARY KEY NONCLUSTERED (dwid_tienda)
)

go

IF OBJECT_ID('DimTienda') IS NOT NULL
    PRINT '<<< CREATED TABLE DimTienda >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimTienda >>>'
go

/*
 * TABLE: DimUbicacion
 */

CREATE TABLE DimUbicacion(
    dwid_ubicacion bigint    NOT NULL,
    ciudad          varchar(30) NOT NULL,
    pais            varchar(20) NOT NULL,
    region          varchar(20) NOT NULL,
    CONSTRAINT PK4 PRIMARY KEY NONCLUSTERED (dwid_ubicacion)
)

go

IF OBJECT_ID('DimUbicacion') IS NOT NULL
    PRINT '<<< CREATED TABLE DimUbicacion >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE DimUbicacion >>>'
go

/*
 * TABLE: HechoContratos
 */

CREATE TABLE HechoContratos(
    dwid_contrato    bigint    NOT NULL,
    id_contrato      bigint    NOT NULL,
    costo_contrato    money     NOT NULL,
    porcentaje_banda  float     NOT NULL,
    porcentaje_manager float     NULL,
    porcentaje_sello  float     NOT NULL,
    dwid_discocontrato bigint    NOT NULL,
    dwid_banda        bigint    NOT NULL,
    dwid_manager       bigint    NULL,
    dwid_tiempo_inicio bigint    NOT NULL,
    dwid_tiempo_fin    bigint    NOT NULL,
    CONSTRAINT PK10 PRIMARY KEY NONCLUSTERED (dwid_contrato)
)

```

```

)

go

IF OBJECT_ID('HechoContratos') IS NOT NULL
    PRINT '<<< CREATED TABLE HechoContratos >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE HechoContratos >>>'
go

/*
* TABLE: HechoProducciones
*/

CREATE TABLE HechoProducciones(
    dwid_produccion    bigint NOT NULL,
    id_produccion      bigint NOT NULL,
    costo_produccion   money  NOT NULL,
    dwid_tiempo_inicio bigint NOT NULL,
    dwid_tiempo_fin    bigint NULL,
    dwid_disco          bigint NOT NULL,
    CONSTRAINT PK8 PRIMARY KEY NONCLUSTERED (dwid_produccion)
)

go

IF OBJECT_ID('HechoProducciones') IS NOT NULL
    PRINT '<<< CREATED TABLE HechoProducciones >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE HechoProducciones >>>'
go

/*
* TABLE: HechoVentas
*/

CREATE TABLE HechoVentas(
    dwid_venta          bigint NOT NULL,
    id_venta            bigint NOT NULL,
    precio_unitario     money  NOT NULL,
    precio_total        money  NOT NULL,
    cantidad_pedida     int    NOT NULL,
    cantidad_vendida    int    NOT NULL,
    hay_demanda_insatisfecha bit NOT NULL,
    dwid_tienda         bigint NOT NULL,
    dwid_disco          bigint NOT NULL,
    dwid_tiempo_pedido  bigint NOT NULL,
    dwid_tiempo_entrega bigint NULL,
    CONSTRAINT PK6 PRIMARY KEY NONCLUSTERED (dwid_venta)
)

go

IF OBJECT_ID('HechoVentas') IS NOT NULL
    PRINT '<<< CREATED TABLE HechoVentas >>>'
ELSE
    PRINT '<<< FAILED CREATING TABLE HechoVentas >>>'

```

go

```
/*  
 * TABLE: DimArtista  
 */
```

```
ALTER TABLE DimArtista ADD CONSTRAINT RefDimBanda16  
  FOREIGN KEY (dwid_banda)  
  REFERENCES DimBanda(dwid_banda)
```

go

```
ALTER TABLE DimArtista ADD CONSTRAINT RefDimInstrumento18  
  FOREIGN KEY (dwid_instrumento)  
  REFERENCES DimInstrumento(dwid_instrumento)
```

go

```
/*  
 * TABLE: DimBanda  
 */
```

```
ALTER TABLE DimBanda ADD CONSTRAINT RefDimArtista33  
  FOREIGN KEY (dwid_representante)  
  REFERENCES DimArtista(dwid_artista)
```

go

```
ALTER TABLE DimBanda ADD CONSTRAINT RefDimManager34  
  FOREIGN KEY (dwid_manager)  
  REFERENCES DimManager(dwid_manager)
```

go

```
ALTER TABLE DimBanda ADD CONSTRAINT RefDimCategoria17  
  FOREIGN KEY (dwid_categoria)  
  REFERENCES DimCategoria(dwid_categoria)
```

go

```
/*  
 * TABLE: DimDisco  
 */
```

```
ALTER TABLE DimDisco ADD CONSTRAINT RefDimBanda11  
  FOREIGN KEY (dwid_banda)  
  REFERENCES DimBanda(dwid_banda)
```

go

```
ALTER TABLE DimDisco ADD CONSTRAINT RefDimDiscoContrato12  
  FOREIGN KEY (dwid_discocontrato)  
  REFERENCES DimDiscoContrato(dwid_discocontrato)
```

go

```
ALTER TABLE DimDisco ADD CONSTRAINT RefDimTiempo24  
  FOREIGN KEY (dwid_tiempo_lanzamiento)  
  REFERENCES DimTiempo(dwid_tiempo)
```

go

```
/*  
 * TABLE: DimTienda  
 */
```

```

ALTER TABLE DimTienda ADD CONSTRAINT RefDimUbicacion4
    FOREIGN KEY (dwid_ubicacion)
    REFERENCES DimUbicacion(dwid_ubicacion)
go

/*
* TABLE: HechoContratos
*/

ALTER TABLE HechoContratos ADD CONSTRAINT RefDimDiscoContrato13
    FOREIGN KEY (dwid_discocontrato)
    REFERENCES DimDiscoContrato(dwid_discocontrato)
go

ALTER TABLE HechoContratos ADD CONSTRAINT RefDimBanda15
    FOREIGN KEY (dwid_banda)
    REFERENCES DimBanda(dwid_banda)
go

ALTER TABLE HechoContratos ADD CONSTRAINT RefDimManager20
    FOREIGN KEY (dwid_manager)
    REFERENCES DimManager(dwid_manager)
go

ALTER TABLE HechoContratos ADD CONSTRAINT RefDimTiempo27
    FOREIGN KEY (dwid_tiempo_inicio)
    REFERENCES DimTiempo(dwid_tiempo)
go

ALTER TABLE HechoContratos ADD CONSTRAINT RefDimTiempo30
    FOREIGN KEY (dwid_tiempo_fin)
    REFERENCES DimTiempo(dwid_tiempo)
go

/*
* TABLE: HechoProducciones
*/

ALTER TABLE HechoProducciones ADD CONSTRAINT RefDimTiempo22
    FOREIGN KEY (dwid_tiempo_inicio)
    REFERENCES DimTiempo(dwid_tiempo)
go

ALTER TABLE HechoProducciones ADD CONSTRAINT RefDimTiempo23
    FOREIGN KEY (dwid_tiempo_fin)
    REFERENCES DimTiempo(dwid_tiempo)
go

ALTER TABLE HechoProducciones ADD CONSTRAINT RefDimDisco26
    FOREIGN KEY (dwid_disco)
    REFERENCES DimDisco(dwid_disco)
go

/*
* TABLE: HechoVentas
*/

ALTER TABLE HechoVentas ADD CONSTRAINT RefDimTienda5
    FOREIGN KEY (dwid_tienda)

```

```
REFERENCES DimTienda(dwid_tienda)
go
```

```
ALTER TABLE HechoVentas ADD CONSTRAINT RefDimDisco7
FOREIGN KEY (dwid_disco)
REFERENCES DimDisco(dwid_disco)
go
```

```
ALTER TABLE HechoVentas ADD CONSTRAINT RefDimTiempo8
FOREIGN KEY (dwid_tiempo_pedido)
REFERENCES DimTiempo(dwid_tiempo)
go
```

```
ALTER TABLE HechoVentas ADD CONSTRAINT RefDimTiempo21
FOREIGN KEY (dwid_tiempo_entrega)
REFERENCES DimTiempo(dwid_tiempo)
go
```